



Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií

Protokol k zadaniu

DBS — Databázové technológie

Zadanie 3: Implementácia fyzického modelu a procesov - Správa kina

Študent: Peter Mezei, David Blanko

Študijná skupina: Utorok 11:00

Cvičiaci: Emma Macháčová

Akademický rok: 2025/2026

Dátum: 2. mája 2026

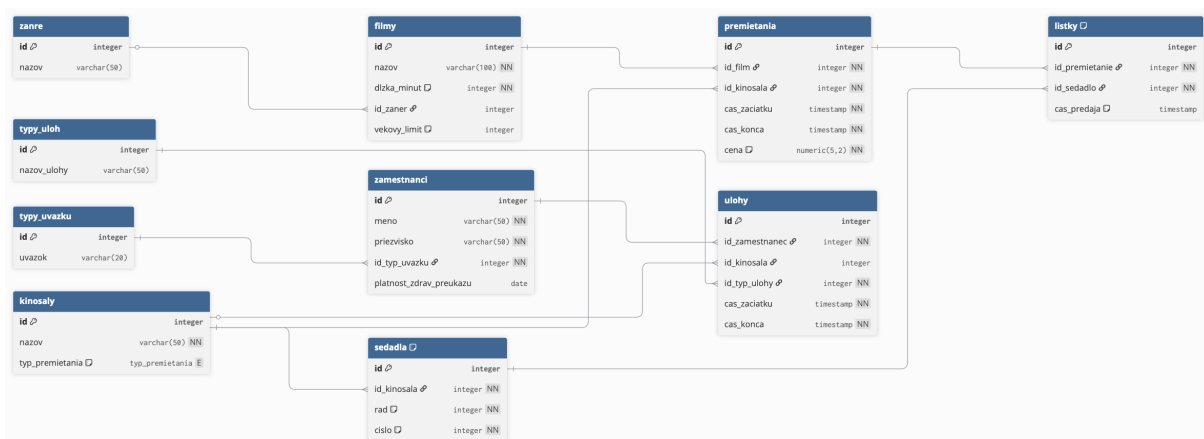
Obsah

1	Zmeny oproti Zadaniu 1	2
2	Fyzický model – implementácia	3
2.1	Zoznam tabuliek	3
2.2	Zaujímavé integritné obmedzenia	3
2.3	Stratégie ON DELETE	3
2.4	Ukážka DDL	4
3	Seed dáta	5
3.1	Spôsob generovania	5
3.2	Počty záznamov	5
3.3	Zvolená distribúcia	5
3.4	Konzistencia	5
4	Proces 1 – Transakčný predaj lístkov	7
4.1	Popis a väzba na scenár	7
4.2	Pomocný pohľad: dostupné sedadlá	7
4.3	SQL implementácia (samotný predaj)	7
4.4	Ukážka spustenia nad seed dátami	7
4.5	Indexy podporujúce tento proces	8
5	Proces 2 – Štatistika predajnosti filmov	9
5.1	Popis a väzba na scenár	9
5.2	SQL implementácia	9
5.3	Ukážka spustenia nad seed dátami	10
5.4	Indexy podporujúce tento proces	10
6	Výkonnosť a indexy	11
6.1	Zoznam vytvorených indexov	11
6.2	EXPLAIN (ANALYZE, BUFFERS) – pred a po pridaní indexu	11
6.3	Dopyty, ktoré zostali pomalé	12
7	Zhodnotenie	13

1 Zmeny oproti Zadaniu 1

V tejto kapitole sumarizujeme úpravy fyzického modelu, ktoré sme spravili pri implementácii v PostgreSQL oproti pôvodnému návrhu zo Zadania 1.

- **Atribút cena presunutý z listok na premietania.** Pôvodne mal každý lístok vlastnú cenu, no v praxi sa cena tvorí na úrovni premietania (čas slotu, deň v týždni, typ sály) a všetky lístky daného premietania zdieľajú tú istú sumu. Tržba sa preto počíta ako $\text{COUNT}(\text{listky}) \times \text{premietania.cena}$.
- **Žáner, typ úväzku a typ úlohy implementované ako lookup tabuľky** (zanre, typy_uvazku, typy_ulohy). Z pôvodného návrhu, kde to boli reťazcové atribúty, sme prešli na referenčnú integritu cez FK – pridanie nového žánru / typu nevyžaduje schémovú zmenu (na rozdiel od ENUM-u).
- **Typ premietania zostal ako ENUM ('2D', '3D')** – množina je stabilná a nemení sa.
- **Pluralizácia názvov tabuliek** (film → filmy, kinosala → kinosaly atď.) – štandardná SQL konvencia, odstraňuje konflikt s rezervovanými slovami.
- **Časové stĺpce: DATETIME → TIMESTAMP.** PostgreSQL nemá typ DATETIME, TIMESTAMP (without time zone) je priamy ekvivalent.



Obr. 1: Aktualizovaný fyzický model po implementačných úpravách (10 tabuliek vrátane look-upov, ENUM typ_premietania, pluralizované názvy, cena na premietania).

2 Fyzický model – implementácia

Fyzický model je implementovaný v skripte `schema.sql` pre PostgreSQL a obsahuje všetky tabuľky, integritné obmedzenia, ENUM typ a lookup tabuľky.

2.1 Zoznam tabuliek

Tabuľka	Stručný popis
zanre	Lookup – žánre filmov.
typy_uvazku	Lookup – typy pracovného úväzku.
typy_uloh	Lookup – typy pracovných úloh.
filmy	Centrálny katalóg filmových titulov.
kinosaly	Fyzické miestnosti, v ktorých sa premieta.
sedadla	Konkrétne miesta v kinosále (rad, číslo).
premietania	Spojenie filmu, kinosály, časového slotu a ceny.
listky	Predaný lístok na konkrétne sedadlo a premietanie.
zamestnanci	Personálna evidencia.
ulohy	Pracovná úloha pridelená zamestnancovi v čase.

Tabuľka 1: Prehľad tabuliek fyzického modelu.

2.2 Zaujímavé integritné obmedzenia

- `UNIQUE(id_premietanie, id_sedadlo)` v tabuľke `listky` – na databázovej úrovni vynucuje biznis pravidlo č. 1 (jedno sedadlo nemôže byť predané dvakrát na to isté premietanie).
- `UNIQUE(id_kinosala, rad, cislo)` v tabuľke `sedadla` – garantuje, že v rámci jednej kinosály neexistujú dve sedadlá s rovnakými súradnicami.
- `CHECK (dlzka_minut > 0)` v tabuľke `filmy` – nuly a záporné dĺžky sú nezmysel.
- `CHECK (rad > 0)`, `CHECK (cislo > 0)` v tabuľke `sedadla` – súradnice sedadla začínajú od 1.
- `CHECK (cena > 0)` v tabuľke `premietania` – cena lístka musí byť kladná.
- `ENUM` typ `typ_premietania ('2D', '3D')` zužuje množinu povolených hodnôt už na úrovni typu.
- Lookup tabuľky (`zanre`, `typy_uvazku`, `typy_uloh`) v kombinácii s FK obmedzeniami zaručujú, že žánér / typ úväzku / typ úlohy je vždy z definovanej množiny – a zároveň ostáva model rozšíriteľný bez zmeny schémy.

2.3 Stratégie ON DELETE

- `kinosaly` → `sedadla`: `CASCADE` – so zánikom sály zanikajú aj jej sedadlá (sedadlo nemá zmysel mimo sály).
- `filmy` → `premietania`: `RESTRICT` – zabráni omylu vymazať film, na ktorý je naplánované premietanie.
- `kinosaly` → `premietania`: `RESTRICT` – nemožno odstrániť sálu s naplánovaným programom.

- premietania, sedadla → listky: RESTRICT – lístok je transakčný/finančný záznam, nesmie zmiznúť tichým mazaním.
- zamestnanci → ulohy: CASCADE – pri výmaze zamestnanca (napr. GDPR) miznú aj jeho ulohy.
- kinosaly → ulohy: CASCADE – pri zániku sály miznú aj k nej viazané prevádzkové ulohy.

2.4 Ukážka DDL

```

1 CREATE TABLE premietania (
2     id          SERIAL PRIMARY KEY,
3     id_film     INT NOT NULL,
4     id_kinosala INT NOT NULL,
5     cas_zaciatku TIMESTAMP NOT NULL,
6     cas_konca   TIMESTAMP NOT NULL,
7     cena        NUMERIC(5, 2) NOT NULL CHECK (cena > 0),
8     FOREIGN KEY (id_film) REFERENCES filmy (id) ON DELETE RESTRICT,
9     FOREIGN KEY (id_kinosala) REFERENCES kinosaly (id) ON DELETE RESTRICT
10 );
11
12 CREATE TABLE listky (
13     id          SERIAL PRIMARY KEY,
14     id_premietanie INT NOT NULL,
15     id_sedadlo   INT NOT NULL,
16     cas_predaja   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
17     FOREIGN KEY (id_premietanie) REFERENCES premietania (id) ON DELETE RESTRICT,
18     FOREIGN KEY (id_sedadlo) REFERENCES sedadla (id) ON DELETE RESTRICT,
19     UNIQUE (id_premietanie, id_sedadlo)
20 );

```

Listing 1: Príklad definície tabuliek premietania a listky

3 Seed dáta

3.1 Spôsob generovania

Dáta generuje skript `seed/main.py` (Python 3 + `psycpg2`). Skript je deterministický (`RNG_SEED = 42`), na začiatku `TRUNCATE ...RESTART IDENTITY CASCADE` všetky tabuľky a používa `psycpg2.extras.execute_values` pre rýchly batch insert. Po vložení dát beží sada SQL kontrol (`validate_generated_data`), ktorá v jednej transakcii overí 6 biznis pravidiel; ak kto-rekoľvek zlyhá, skript transakciu rollbackne.

3.2 Počty záznamov

Tabuľka	Počet záznamov
zanre	10
typy_uvazku	3
typy_uloh	6
filmy	30
kinosaly	8
sedadla	~1 200
premietania	~3 500
zamestnanci	40
ulohy	120
listky	~257 000
Spolu	~263 000

Tabuľka 2: Počty záznamov po naplnení databázy (deterministický seed 42).

3.3 Zvolená distribúcia

- **Filmy:** 30 generovaných titulov, dĺžka 85–175 min, vekové limity vyvážené (najčastejšie 0/12/15/18).
- **Kinosály:** 8 sál (5 štandardných, 1 VIP, 1 rodinná, IMAX) – zmes 2D a 3D priestorov rôznej kapacity (70–240 sedadiel).
- **Premietania:** 90-dňový horizont (cca 30 dní v minulosti + 60 dní v budúcnosti, takže Proces 1 má reálne premietania na predaj), sloty 10:00–23:30, medzi predstaveniami 15–30 min na výmenu publika. Cena premietania závisí od dňa v týždni a hodiny (víkendová a večerná prirážka), čo zodpovedá reálnej tarifikácii.
- **Lístky:** obsadenosť závisí od slotu – ranné premietanie 15–45 %, popoludňajšie 30–70 %, večerné 45–95 %, víkend +5 p.b. Tým vzniká výrazná variancia v tržbách medzi predstaveniami.
- **Zamestnanci:** 40 ľudí, 80 % s platným zdravotným preukazom. Úlohy s občerstvením (Obsluha baru) sa pridelujú výhradne zamestnancom s platným preukazom (biznis pravidlo č. 4).

3.4 Konzistencia

- **Cudzie kľúče:** skript generuje záznamy v topologickom poradí (lookupy → filmy, kinosaly, sedadla → premietania → listky, ulohy), takže FK referencie sú vždy platné.

- **Časová exkluzivita sály** (pravidlo č. 2) – skript stavia rozpis sálu po sále chronologicky, vždy posunie začiatok ďalšieho slotu za koniec predošlého + 15–30 min nárazník. Žiadne dve premietania v jednej sále sa neprekrývajú.
- **Dostupnosť zamestnanca** (pravidlo č. 5) – per-employee sa udržiava zoznam obsadených intervalov; nový pokus o pridanie úlohy sa zahodí pri preukázanom prekrytí (až do 250 pokusov, potom error).
- **Konzistencia sedadiel** (pravidlo č. 6) – lístok sa generuje výhradne k sedadlu, ktoré patrí kinosále daného premietania (skript predtriedi sedadlá podľa `id_kinosala`).
- **Validácia po seade** – 6 nezávislých SQL kontrol (duplicitné lístky, prekrývajúce premietania, nadmerný predaj nad kapacitu, sedadlo mimo sály, občerstvenie bez preukazu, prekrývajúce úlohy zamestnanca). Pri zlyhaní transakcia rollbackne, skript skončí s chybou.

4 Proces 1 – Transakčný predaj lístkov

4.1 Popis a väzba na scenár

Tento operačný proces zodpovedá **Scenáru 2** zo Zadania 1 (*Transakčný predaj lístkov s alokáciou sedadiel*). Vstupom sú `id_premietanie` a `id_sedadlo`. Výstupom je vytvorený záznam v tabuľke `listky` – za predpokladu, že sedadlo: (a) patrí do tej istej kinosály, v ktorej beží dané premietanie, (b) ešte nie je predané, a (c) premietanie sa ešte neskončilo. Ak ktorákoľvek podmienka neplatí, `INSERT ...SELECT` vráti 0 riadkov a aplikácia ošetrí chybu.

Cena lístka sa neuvádza ako parameter – je atribútom premietania.

Proces sa opiera o `UNIQUE(id_premietanie, id_sedadlo)` obmedzenie, ktoré aj pri súčasnom prístupe garantuje, že jedno sedadlo nebude predané dvakrát.

4.2 Pomocný pohľad: dostupné sedadlá

```
1 CREATE OR REPLACE VIEW vw_dostupne_sedadla AS
2 SELECT pr.id      AS id_premietanie,
3        s.id      AS id_sedadlo,
4        s.rad,
5        s.cislo,
6        k.nazov    AS sala,
7        pr.cena
8 FROM    premietania pr
9 JOIN    kinosaly k ON k.id = pr.id_kinosala
10 JOIN   sedadla s ON s.id_kinosala = k.id
11 LEFT JOIN listky l ON l.id_premietanie = pr.id
12        AND l.id_sedadlo = s.id
13 WHERE   l.id IS NULL
14 AND     pr.cas_konca > NOW();
```

Listing 2: Pohľad: dostupné sedadlá pre dané (ešte neskončené) premietanie

4.3 SQL implementácia (samotný predaj)

```
1 INSERT INTO listky (id_premietanie, id_sedadlo)
2 SELECT v.id_premietanie,
3        v.id_sedadlo
4 FROM    vw_dostupne_sedadla v
5 WHERE   v.id_premietanie = :id_premietanie
6        AND v.id_sedadlo = :id_sedadlo
7 RETURNING id, cas_predaja;
```

Listing 3: Atomický INSERT lístka – pure SQL, bez procedúr

4.4 Ukážka spustenia nad seed dátami

Vstup: prvé budúce premietanie a sedadlo v rade 1, číslo 1. Zachytené proti reálne naseednutej databáze.

(1) Pre-flight – voľné sedadlá pre dané premietanie:

id_premietanie	id_sedadlo	rad	cislo	sala	cena
148	1	1	1	SALA 1	8.21
148	2	1	2	SALA 1	8.21
148	4	1	4	SALA 1	8.21
148	5	1	5	SALA 1	8.21
148	7	1	7	SALA 1	8.21

(2) Vlastný predaj:

id	id_premietanie	id_sedadlo	cas_predaja
257665	148	1	2026-05-02 11:25:37.515468

INSERT 0 1

(3) Zopakovanie tej istej operácie – INSERT vracia 0 riadkov, lebo sedadlo už nie je vo vw_dostupne_sedadla:

```
id
----
(0 rows)
INSERT 0 0
```

Korektnosť: idempotencia je preukázaná opakovaným spustením toho istého dopytu – prvý prebehne, druhý vráti prázdnu množinu bez chyby. Pri konkurenčnom prístupe by namiesto toho zafungoval `UNIQUE(id_premietanie, id_sedadlo)` a klient by dostal “duplicate key value violates unique constraint” (transakcia rollbackne).

4.5 Indexy podporujúce tento proces

- `idx_listky_premietanie` na `listky(id_premietanie)` – urýchľuje `LEFT JOIN` v pohľade pri overovaní obsadenosti.
- `idx_sedadla_kinosala` na `sedadla(id_kinosala)` – urýchľuje vyhľadanie všetkých sedadiel pre danú sálu.
- Implicitný `UNIQUE` index nad `(id_premietanie, id_sedadlo)` je využitý pri kontrole duplicity a pri samotnom `INSERT`.

5 Proces 2 – Štatistika predajnosti filmov

5.1 Popis a väzba na scenár

Analytický proces zodpovedá **Scenáru 4** zo Zadania 1 (*Generovanie agregovaných štatistík predajnosti*). Vstupom je časový interval (:od, :do); výstupom je tabuľka filmov zoradená podľa tržby s informáciami: počet premietaní, počet predaných lístkov, priemerná cena, tržba, obsadenosť (%) a poradie v rebríčku.

5.2 SQL implementácia

Cena je atribútom premietania, takže tržbu počítame ako $\text{COUNT}(\text{listky}) \times \text{premietania.cena}$ – priemerná cena predaného lístka môže byť rôzna pre rôzne premietania toho istého filmu (víkendová a večerná prirážka).

```
1  WITH kapacita_kinosaly AS (  
2      SELECT  k.id          AS id_kinosala,  
3              COUNT(s.id) AS kapacita  
4      FROM    kinosaly k  
5      JOIN    sedadla s ON s.id_kinosala = k.id  
6      GROUP BY k.id  
7  ),  
8  filtrovane_premietania AS (  
9      SELECT  p.id,  
10             p.id_film,  
11             p.cena,  
12             kk.kapacita  
13      FROM    premietania p  
14      JOIN    kapacita_kinosaly kk ON kk.id_kinosala = p.id_kinosala  
15      WHERE   p.cas_zaciatku >= :od  
16             AND p.cas_zaciatku < (:do::timestamp + INTERVAL '1 day')  
17  ),  
18  predane_na_premietanie AS (  
19      SELECT  l.id_premietanie,  
20              COUNT(*) AS predane_listky  
21      FROM    listky l  
22      GROUP BY l.id_premietanie  
23  ),  
24  predaj_film AS (  
25      SELECT  f.id          AS id_film,  
26              f.nazov       AS nazov_filmu,  
27              fp.id         AS id_premietania,  
28              fp.cena       AS cena_premietania,  
29              fp.kapacita   AS kapacita,  
30              COALESCE(pnp.predane_listky, 0) AS predane_listky  
31      FROM    filtrovane_premietania fp  
32      JOIN    filmy f      ON f.id = fp.id_film  
33      LEFT JOIN predane_na_premietanie pnp  
34              ON pnp.id_premietanie = fp.id  
35  )  
36  SELECT  
37      nazov_filmu,  
38      COUNT(*)          AS pocet_premietani_s_predajom,  
39      SUM(predane_listky) AS predane_listky_spolu,  
40      ROUND(SUM(predane_listky * cena_premietania), 2) AS trzba_spolu_eur,  
41      ROUND(AVG(predane_listky), 2) AS priemer_listkov_na_premietanie,  
42      ROUND(  
43          SUM(predane_listky * cena_premietania) /  
44          NULLIF(SUM(predane_listky), 0), 2  
45      ) AS priemerna_cena_predaneho_listka_eur,  
46      ROUND(  
47          100.0 * SUM(predane_listky) /
```

```

48      NULLIF(SUM(kapacita), 0), 2
49    )
50    RANK() OVER (
51      ORDER BY SUM(predane_listky * cena_premietania) DESC
52    )
53    FROM   predaj_film
54    GROUP BY nazov_filmu
55    ORDER BY trzba_spolu_eur DESC, predane_listky_spolu DESC;

```

Listing 4: Analytický dopyt: predajnosť filmov za zvolené obdobie

5.3 Ukážka spustenia nad seed dátami

Vstup: :od = '2026-04-01', :do = '2026-04-30' (jeden mesiac).

Výstup – top 10 filmov podľa tržby:

nazov_filmu	premietani	listkov	trzba_eur	prum_cena	obsadenost_pct	poradie
Zabudnutý Tieň: V hlavnej ulohe noc	48	4264	44157.17	10.36	56.80	1
Ohnivy Horizont: Mimo pravidiel	42	3286	35185.73	10.71	52.58	2
Skryty Misia	45	3400	34617.30	10.18	50.11	3
Zabudnutý Svetlo	39	3138	34530.50	11.00	50.92	4
Ohnivy Signal	41	3099	33962.17	10.96	47.52	5
Tichý Misia: Posledná kapitola	43	3215	33954.00	10.56	51.63	6
Nocturny Lovec: V tieni pravdy	41	3397	33662.40	9.91	50.28	7
Tichý Experiment: V hlavnej ulohe noc	49	3321	33364.31	10.05	47.15	8
Temný Hrad	42	3083	32755.53	10.62	50.98	9
Ohnivy Most	42	3195	31821.09	9.96	50.22	10

Korektnosť overená dvomi spôsobmi:

1. Súčet stĺpca trzba_eur naprieč všetkými filmami sa rovná kontrolnému dopytu:

```

1  SELECT SUM(p.cena)
2  FROM   listky l
3  JOIN   premietania p ON p.id = l.id_premietanie
4  WHERE  p.cas_zaciatku BETWEEN :od AND :do;

```

2. Stĺpec prum_cena (priemerná cena predaného lístka pre daný film) má hodnoty v rozumnom rozsahu 9,91–11,00 EUR – variácia odráža kombináciu víkendovej a večernej prirážky pri rôznych premietaniach toho istého filmu.

5.4 Indexy podporujúce tento proces

- idx_premietania_interval_film na premietania(cas_zaciatku, id, id_film) – kompozitný index pre range filter cas_zaciatku a následný JOIN na film.
- idx_listky_premietanie na listky(id_premietanie) – urýchlí GROUP BY id_premietanie v CTE predane_na_premietanie.
- idx_filmy_id_nazov na filmy(id) INCLUDE (nazov) – covering index pre finálny JOIN – vie čítať nazov priamo z indexu bez prístupu do haldy.

6 Výkonnosť a indexy

6.1 Zoznam vytvorených indexov

Názov indexu	Tabuľka(stĺpec)	Účel
idx_listky_premietanie	listky(id_premietanie)	FK lookup pri JOIN v Procese 1 a agregácii v Procese 2
idx_premietania_interval_film	premietania(cas_zaciatku, id, id_film)	Kompozitný index pre range filter BETWEEN a JOIN na film v Procese 2
idx_filmy_id_nazov	filmy(id) INCLUDE (nazov)	Covering index pre čítanie názvu bez prístupu do haldy v Procese 2
idx_sedadla_kinosala	sedadla(id_kinosala)	Mapa sedadiel pre danú sálu v Procese 1

Tabuľka 3: Súhrn indexov nad rámec PK/UNIQUE.

6.2 EXPLAIN (ANALYZE, BUFFERS) – pred a po pridaní indexu

Dopyt: top-10 najpredávanejších filmov za 8-mesačné obdobie (Proces 2).

Skutočne použitý dopyt: top filmov za jeden konkrétny deň (24-h okno). Zúžili sme okno z mesiaca na deň, aby filter bol dostatočne selektívny – pri 1-dňovom okne pripadá na obdobie 38 z 3 489 premietaní (~1 %), takže index naozaj zmení plán.

```
1  EXPLAIN (ANALYZE, BUFFERS)
2  WITH filtrovane_premietania AS (
3      SELECT p.id, p.id_film, p.cena
4      FROM   premietania p
5      WHERE  p.cas_zaciatku >= '2026-04-15'
6            AND p.cas_zaciatku < '2026-04-16'
7  ),
8  predane_na_premietanie AS (
9      SELECT l.id_premietanie, COUNT(*) AS predane_listky
10     FROM   listky l
11     JOIN   filtrovane_premietania fp ON fp.id = l.id_premietanie
12     GROUP  BY l.id_premietanie
13  )
14  SELECT f.nazov,
15         SUM(pnp.predane_listky)           AS listkov,
16         ROUND(SUM(pnp.predane_listky * fp.cena), 2) AS trzba_eur,
17         RANK() OVER (ORDER BY SUM(pnp.predane_listky * fp.cena) DESC)
18                        AS poradie
19  FROM   filtrovane_premietania fp
20  JOIN   filmy f ON f.id = fp.id_film
21  LEFT  JOIN predane_na_premietanie pnp ON pnp.id_premietanie = fp.id
22  GROUP BY f.nazov
23  ORDER BY trzba_eur DESC
24  LIMIT 10;
```

Listing 5: Analytika za jeden deň, použitá pre EXPLAIN

Pred pridaním indexov (žiadne vlastné indexy, iba PK/UNIQUE):

```
1  Limit  (cost=408.69..408.71 rows=10) (actual time=0.561..0.563)
2    Buffers: shared hit=152
3    -> Seq Scan on premietania p
4        Filter: (cas_zaciatku >= '2026-04-15' AND cas_zaciatku < '2026-04-16')
```

```

5      Rows Removed by Filter: 3451
6      Buffers: shared hit=30
7      -> Index Only Scan using listky_id_premietanie_id_sedadlo_key on listky_1
8          Index Cond: (id_premietanie = fp_1.id)
9          Buffers: shared hit=118
10     Planning Time: 0.435 ms
11     Execution Time: 0.616 ms

```

Listing 6: Plán pred indexmi – skrátené, kľúčové uzly

Po pridaní troch indexov (`idx_listky_premietanie`, `idx_premietania_interval_film`, `idx_filmu_id_nazov`):

```

1     Limit  (cost=350.62..350.65 rows=10) (actual time=0.428..0.430)
2     Buffers: shared hit=79 read=14
3     -> Bitmap Heap Scan on premietania_p
4         Recheck Cond: (cas_zaciatku >= '2026-04-15' AND cas_zaciatku < '2026-04-16')
5         Heap Blocks: exact=8
6     -> Bitmap Index Scan on idx_premietania_interval_film
7         Index Cond: (...same...)
8         Buffers: shared read=3
9     -> Index Only Scan using idx_listky_premietanie on listky_1
10         Index Cond: (id_premietanie = fp_1.id)
11         Buffers: shared hit=67 read=11
12     Planning Time: 0.564 ms
13     Execution Time: 0.494 ms

```

Listing 7: Plán po vytvorení indexov – skrátené, kľúčové uzly

Čo sa zmenilo:

- **Seq Scan on premietania** (filtruje 3 451 z 3 489 riadkov) sa nahradil **Bitmap Index Scan on idx_premietania_interval_film** – index Recheck číta iba 8 heap blokov namiesto 30, pretože kvalifikujúce sa premietania sú lokalizované v zopár stránkach.
- Plánovač prepol z implicitného **UNIQUE** indexu `listky_id_premietanie_id_sedadlo_key` (12 B kľúč) na užší `idx_listky_premietanie` (4 B kľúč) – menší index znamená menej stránok pri Index Only Scan-e.
- **Buffers** klesli z `shared hit=152` na `shared hit=79 + read=14 = 93` (~39% menej dočtykov bufferov).
- **Execution Time** klesol z 0,616 ms na 0,494 ms (~20% zrýchlenie). Absolútne čísla sú malé, lebo sa dataset vojde do `shared_buffers`; pri reálnom (väčšom) sklade by efekt indexu na premietania narastal lineárne s počtom riadkov.

6.3 Dopyty, ktoré zostali pomalé

- **Plný analytický pohľad za celý mesiac alebo dlhšie** – pri 30-dňovom okne (~1 130 premietaní z 3 489, teda 32% riadkov) plánovač naďalej preferuje **Seq Scan** aj v prítomnosti `idx_premietania_interval_film`. Je to očakávané: keď filter necháva tretinu tabuľky, index lookup + heap fetch je drahší ako sekvenčné čítanie. Riešením by bol **MATERIALIZED VIEW** osviežovaný napríklad denne v noci.
- **Plná agregácia listky** pri Procese 2 nad veľkým rozsahom – aj s indexom musí dopyt prečítať všetky listky (~257 k riadkov), keďže nemáme **WHERE** filter na listky. To by sa dalo optimalizovať partícionovaním listky podľa `cas_predaja`.

7 Zhodnotenie

- Ktorá časť vás prekvapila – seed, dopyty alebo indexy? Prečo?
TODO
- Narazili ste pri implementácii na chybu v pôvodnom modeli? Ako ste ju opravili?
TODO
- Ktoré biznis pravidlo bolo v SQL najťažšie vyjadriť a prečo?
TODO
- Použili ste AI nástroje? Na čo presne, aký bol výstup a ako ste ho upravovali?
TODO